✅02：Volatile变量的特性

特性

禁止指令重排列规则

volatile不适合场景

不适合复合操作

特性

保证可见性，不保证原子性：

- 在写一个Volatile变量时，JMM会把该线程本地内存中的变量强制刷新到主内存中去
- 这个写操作会导致其他线程中的Volatile变量缓存无效

禁止指令重排列

禁止指令重排列规则

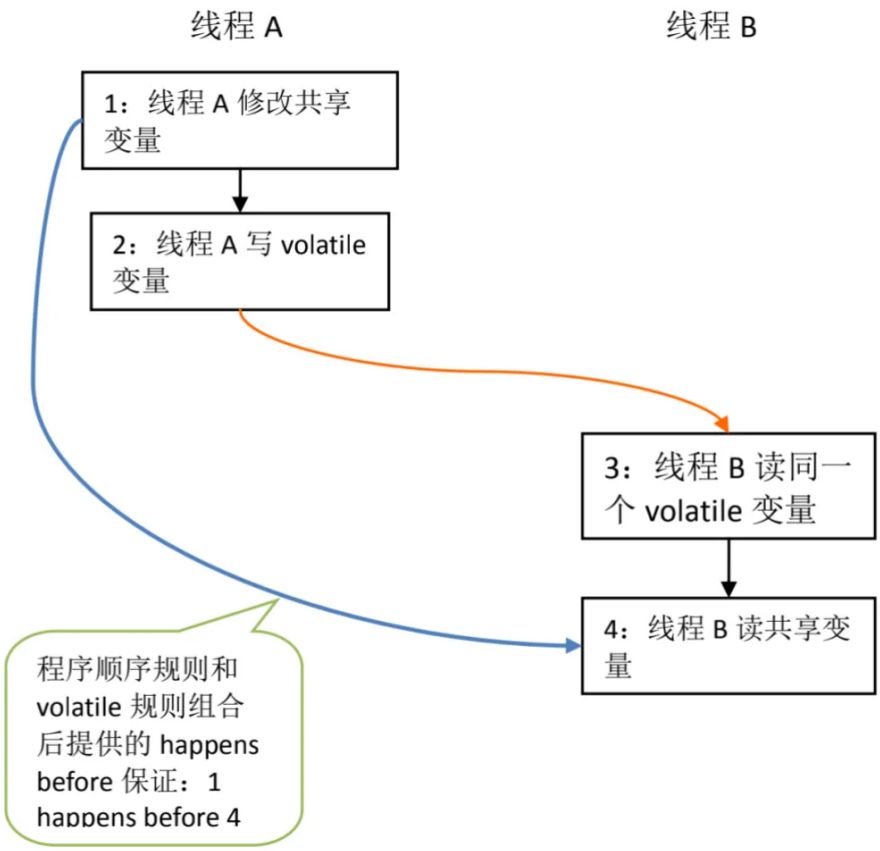
使用volatile关键字修饰共享变量便可以禁止这种重排序。若用volatile修饰共享变量，在编译时，会在指令序列中插入内存屏障来禁止特定类型的处理器重排序。

volatile禁止指令重排序的规则：

- 当程序执行到volatile变量的读操作或者写操作时，在其前面的操作的更改肯定全部已经进行，且结果已经对后面的操作可见；在其后面的操作肯定还没有进行 （处理器阶段）
- 在进行指令优化时，不能将对volatile变量访问的语句放置其后面执行，也不能把volatile变量后面的语句放到其前面执行 (编译器阶段)

简言之，执行到volatile变量时，其前面的所有语句都执行完，后面所有语句都未执行。且前面语句的结果对volatile变量及其后面语句可见。

```java
class ReorderExample {
    int a = 0;
    boolean volatile flag = false;

    public void write() {
        a = 1;              // 1
        flag = true;        // 2

        log.info("{}", i);
    }

    public void read() {
        if (flag) {         //3
            int i = a * a;  // 4

        }

        log.info("{}", i);
    }
}
```



在上图中，每一个箭头链接的两个节点，代表了一个hanppens before关系。黑色箭头标识程序顺序；橙色箭头标识volatile规则；蓝色箭头标识组合这些规则后提供的happens before保证。

这里A线程写一个volatile变量后，B线程读同一个volatile变量。A线程在写volatile变量之前所有可见的共享变量，在B线程读同一个volatile变量后，将立即变得对B线程可见。

volatile不适合场景

不适合复合操作

```java
public class VolatileIncrease {

    public volatile int inc = 0;

    public void increase() {
        inc++; // 非原子操作
    }
}

```

1. 创建10个线程，每个线程执行1000此increase操作
2. 等待所有线程结束。其中Thread.activeCount()来检查当前活跃线程的数量。 > 2的目的是确保除了主线程和测试线程之外，没有其他线程再运行。当Thread.activeCount()减少到2时，意味着所有其他线程都已经结束。

inc++不是一个原子性操作，可以由读取、加、赋值三步组成，所以结果并不能达到10000.

Synchronized

Java

```java
public class SynchronizedIncrease {

    public int inc = 0;

    public synchronized void increase() {
        inc++;
    }
}
```

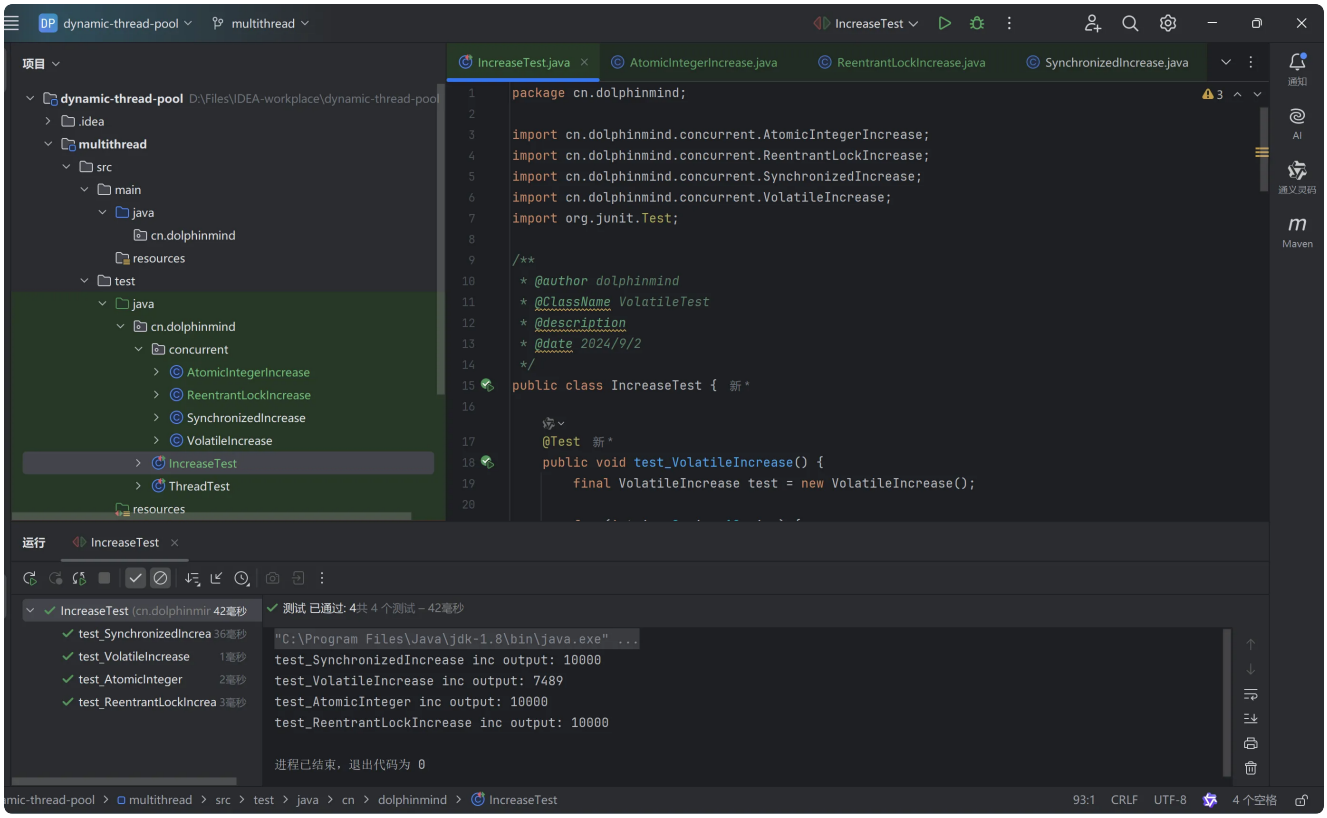
Java

```java
public class ReentrantLockIncrease {

     public int inc = 0;

     Lock lock = new ReentrantLock();

     public void increase() {
         lock.lock();
         inc++;
         lock.unlock();
     }
 }
```

AtomicInteger

Java

```java
public class AtomicIntegerIncrease {

    public AtomicInteger inc = new AtomicInteger();

    public void increase() {
        inc.getAndIncrement();
    }

}
```



双重检查锁定(DCL)单例模式 Java

```java
public class Singleton {

    private static volatile Singleton instance;

    private Singleton() {
        // 私有构造函数，防止外部实例化
    }

    public static Singleton getInstance() {
        if (instance == null) { //第一次检查
            synchronized (Singleton.class) {
                if (instance == null) { // 第二次检查
                    instance = new Singleton(); // 创建单例对象
                }
            }
        }

        return instance;
    }
}
```

双重检查锁定这个名字来源于其实现方式中的两个检查步骤。在DCL单例模式中，双重检查指的时在获取单例对象时进行两次检查，以确保线程安全的同时尽可能减少同步操作带来的性能影响。

第一次检查：

1. 在进入同步块之前，首先检查单例对象是否已经被初始化
2. 如果对象已经被初始化(即instance 不为 null)，则直接返回该对象
3. 如果对象尚未初始化(即instance为null)，则进入同步块

第二次检查：

1. 在同步块内再次检查单例对象是否已经被初始化
2. 如果对象已经被初始化，则直接返回该对象
3. 如果对象尚未初始化，则创建一个新的实例，并将其赋值给Instance

DCL中synchronized在其中的作用

1. 线程安全
    a. 多线程环境下的竞争：在多线程环境下，多个线程可能几乎同时到达instance == null 的检查点。如果没有锁定多个线程可能会同时进入 new Singleton() 的创建过程，从而导致多个实例被创建
    b. 通过synchronized关键字，确保在instance被创建时只有一个线程能够进入该代码块。这样可以保证单例对象的唯一性
2. 初始化过程的原子性
    a. 对象创建过程：在Java中，对象的创建过程分为三步：
        i. 分配内存空间
        ii. 初始化对象
        iii. 设置对象引用
    b. 潜在问题：如果多个线程几乎同时达到 new Singleton()，可能会出现
        i. 线程A分配内存空间，但尚未初始化对象
        ii. 线程B读取到这个尚未初始化的对象引用
    c. 通过synchronized关键字，确保对象的创建过程在一个原子操作中完成，从而避免上述问题

DCL中volatile在其中的作用

1. 可见性：

   a. volatile关键字确保了当一个线程修改了volatile修饰的变量后，其他线程能够立即看到这个变化。这对于单例模式来说十分重要。因为在多线程环境中，我们需要确保当一个线程完成了单例对象的创建之后，其他线程能够立即知道这一点，并能使用已经创建好的单例对象，而不是重复创建。
2. 有序性：
   a. volatile关键字通过在读写操作中插入内存屏障来禁止指令重排序。在Java中，对象的构造过程可能会被编译器或处理器重排序。例如，当一个对象被创建时，内存分配和初始化操作可能会被重排序。如果一个线程在对象尚未完全初始化的情况下就将其引用设置为非空，并且另一个线程督导了这个未完全初始化的对象，就会导致不可预测的行为。volatile关键字确保了对象的初始化过程不会被重排序，从而保证了对象状态的一致性。
3. 原子性：
   a. 尽管volatile不能保证符合操作的原子性，但对于单例模式中的双重检查锁定模式而言，volatile保证了对单例对象引用的赋值操作是原子的。一旦一个线程完成了对单例对象的初始化并将引用赋值给了volatile字段，其他线程就能看到这个变化，并且这个引用不会被重排序。

Java

```java
// instance = new Singleton() 过程可以被拆分成三步

memory = allocate() // 分配内存
ctorInstanc(memory) // 初始化对象
instance = memory   // 设置instance指向刚分配的地址

// 如果没有volatile关键字，上面的操作可能无序
```

简言之，双重检查，第一层减少不必要的同步操作，提高性能，第二层利用sychronized同步代码块创建对象，保证同步，而volatile保证同步代码块内部对象创建引用赋值操作的原子性已经避免重排列引发的问题，一旦创建立即被其他线程看到。